



# **Aplicação de Gestão de Unidade de Saúde**

## **Aplicações Distribuídas** **Mestrado em Informática médica**

**Braga, novembro 2023**

**Docente: António Luís Sousa**

Mariana Ribeiro, pg54061;

Mariana Almeida, pg54062;

Madalena Passos, pg54023.

## Índice

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 1. Introdução .....                                                              | 3  |
| 2. Arquitetura da Aplicação.....                                                 | 4  |
| 3. Definição das Interfaces dos Utilizadores .....                               | 5  |
| 4. Implementação da Aplicação .....                                              | 10 |
| 4.1. Implementação das classes Utente, Médico, Rececionista e Ficha Médica ..... | 10 |
| 4.2. Implementação das Interfaces – Gestor Clínico.....                          | 11 |
| 4.3. Carregamento de Dados .....                                                 | 13 |
| 5. Interação Servidor – Cliente.....                                             | 14 |
| 6. Conclusão .....                                                               | 18 |

## 1. Introdução

Atualmente, os sistemas distribuídos têm desempenhado um papel crucial no desenvolvimento de sistemas de software, promovendo a colaboração eficiente entre diferentes entidades.

Assim, no âmbito da disciplina de Aplicações Distribuídas, este trabalho centra-se na implementação de uma aplicação que pretende realizar a gestão e registo de cuidados de saúde de uma unidade de saúde. A aplicação desenvolvida conta com três tipos distintos de utilizadores, com funções e autorizações de utilização diferentes.

Para uma maior facilidade de implementação da aplicação e todos os seus constituintes foi utilizada a tecnologia Java RMI (*Remote Method Invocation*). Esta tecnologia proporciona uma forma transparente e eficiente de comunicação entre os objetos distribuídos desenvolvidos. Esta abordagem através de Java RMI têm a capacidade de invocar métodos em objetos remotos, como se estivessem presentes no dispositivo local, simplificando a complexidade associada à comunicação distribuída.

Este trabalho explorará a implementação prática da aplicação distribuída, abordando aspetos como a arquitetura geral, o papel específico de cada tipo de utilizador e a interação entre os componentes distribuídos.

No final deste trabalho, espera-se obter uma compreensão aprofundada das aplicações distribuídas e da tecnologia Java RMI, destacando, no entanto, futuros desafios para a melhoria da aplicação.

## 2. Arquitetura da Aplicação

Numa fase inicial do desenvolvimento de um sistema de gestão e registo dos cuidados de saúde e dados fisiológicos de uma unidade de cuidados de saúde, foram definidas 10 entidades: *Utente*, *ProfissionalDeSaude*, *Familiar*, *RececionistaHospitalar*, *Medico*, *FichaMedica*, *Exame*, *Consulta*, *DadosFisiologicos*, *Prescricao*, para posterior implementação, conforme ilustrado na Figura 1.

A arquitetura proposta irá utilizar Java RMI (*Remote Method Invocation*) como tecnologia para permitir a comunicação entre clientes/servidor. Neste sentido, existirão 4 clientes diferentes (*Utente*, *Familiar*, *Medico* e *RececionistaHospitalar*) que irão comunicar com o servidor através de 4 interfaces, também elas diferentes, onde se definirá os métodos e funcionalidades, disponibilizadas no servidor, a que cada um dos clientes terá acesso.

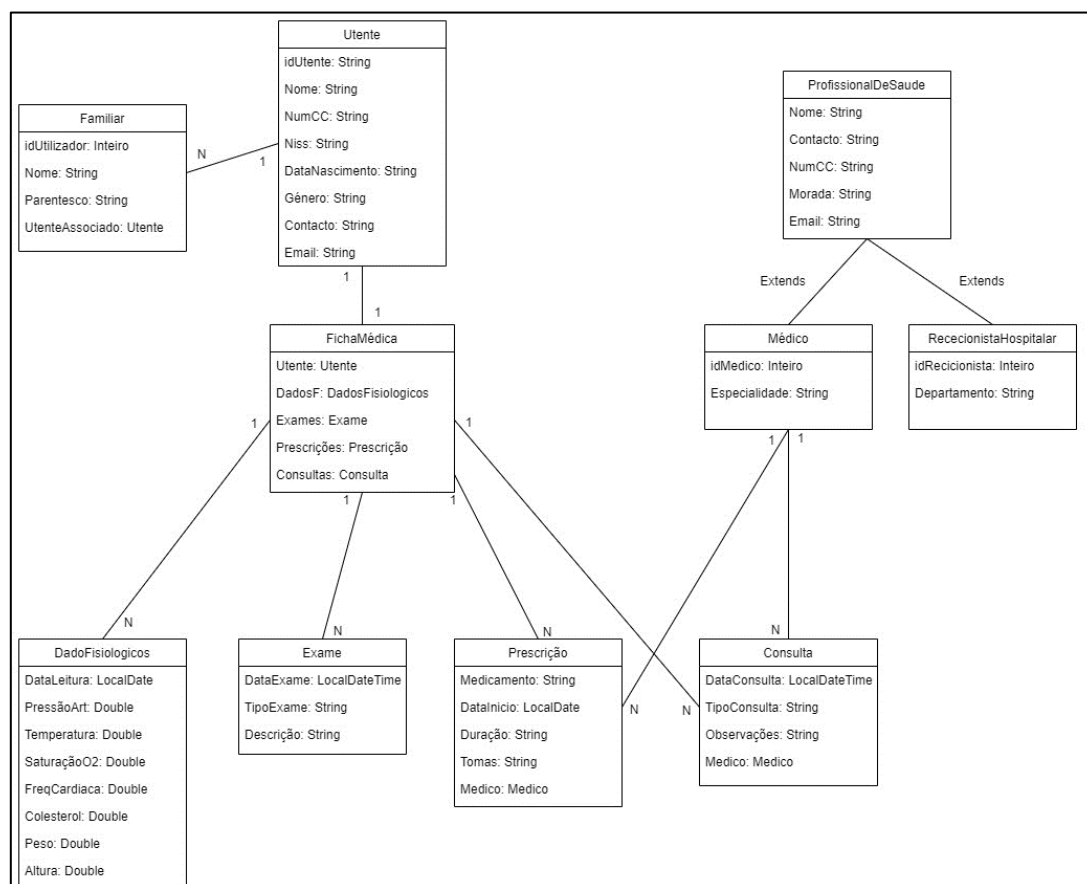


Figura 1- Representação esquemática das entidades definidas.

Através da análise da Figura 1, existem várias conclusões importantes a retirar. No que diz respeito às relações entre entidades:

- Um **Familiar** está associado a um único **Utente**, mas mais de um **Familiar** pode estar associado ao mesmo **Utente**.
- Cada **Ficha Médica** está vinculada a um único **Utente**, e da mesma forma, cada **Utente** possui apenas uma **Ficha Médica** associada.
- Uma **Ficha Médica** pode conter múltiplas leituras de **Dados Fisiológicos**, **Exames**, **Prescrições e Consultas**. No entanto, cada leitura específica desses dados só pode estar associada a uma única **Ficha Médica**.
- Quanto às **Consultas**, um **Médico** pode realizar várias delas, mas cada leitura específica de uma **Consulta** está associada exclusivamente a um único **Médico**.

### 3. Definição das Interfaces dos Utilizadores

Como já referido, o sistema desenvolvido será utilizado por 4 tipos de utilizadores: médicos, rececionistas hospitalares, utentes e familiares dos utentes. Os profissionais de saúde terão permissões para consultar e atualizar os dados dos utentes e respetivas fichas médicas associadas, enquanto os utentes e familiares têm apenas permissão de visualização.

Com efeito, foram desenvolvidas 4 interfaces, onde se encontram métodos específicos que se traduzem em funcionalidades a que um dado utilizador terá acesso. Estes métodos serão implementados posteriormente num **GestorClinico** que será abordado na secção seguinte deste documento. A seguir, mostram-se as definições das 4 interfaces referidas.

Com efeito, a interface do **Utente** contém métodos que permitem:

- Consultar as suas informações pessoais;
- Consultar as suas prescrições, exames, consultas, dados fisiológicos;
- Consultar a respetiva Ficha Médica, caso pretenda consultar todo o seu histórico clínico de uma só vez;
- Consultar todos os seus Familiares associados.

```
public interface UtenteInterface extends Remote {  
  
    public String consultarUtente(int idUtente) throws  
RemoteException;  
    public String consultarConsultaUtente(int idUtente) throws
```

```
RemoteException;  
    public String consultarPrescricoesUtente(int idUtente) throws  
Remote Exception;  
    public String consultarExamesUtente(int idUtente) throws  
RemoteException;  
    public String consultarDadosFisiologicosUtente(int  
idUtente) throws RemoteException;  
    public String consultarFMedicaUtente(int idUtente) throws  
RemoteException;  
  
    public String listarFamPUT (int idUtente) throws RemoteException  
}
```

*Figura 2-Interface do Utente.*

Quanto à interface do **Familiar**, esta contém as seguintes funcionalidades:

- Consultar as informações pessoais do Utente a ele associado;
- Consultar a Ficha Médica do utente associado, que fornecerá todo o seu histórico (Exames, Consultas, Prescrições, Dados Fisiológicos);
- Consultar todas as Consultas do utente a ele associado, para o caso de o familiar apenas pretender visualizar as consultas do utente.

```
public interface FamiliarInterface extends Remote {  
  
    public String familiarConsultaUt(int idFamiliar, int idUtente)  
throws RemoteException;  
  
    public String familiarConsultaFMed(int idFamiliar, int idUtente)  
throws RemoteException;  
  
    public String familiarConsultaConsultas(int idFamiliar, int  
idUtente) throws RemoteException;  
  
}
```

*Figura 3- Interface do Familiar.*

Relativamente à interface **Médico**, esta contém as seguintes funcionalidades:

- Salvar as alterações efetuadas;
- Adicionar leituras de Dados Fisiológicos, Exames, Prescrições e Consultas na Ficha Médica de um dado utente;
- Consultar informações pessoais de um determinado utente;
- Consultar histórico de Exames, Dados Fisiológicos, Consultas e Prescrições de um determinado utente;
- Consultar Ficha Médica de um determinado utente;

- Editar Consultas, Exames e Prescrições de um determinado utente;
- Consultar todos os Utentes, Familiares e Fichas Médicas existentes no sistema, bem como todas os Dados Fisiológicos, Exames, Prescrições e Consultas;
- Consultar o nome de todos os medicamentos já prescritos;
- Consultar todos os Familiares de um determinado Utente;
- Consultar o número total de consultas efetuadas na clínica.
- Consultar o número total de consultas realizadas num determinado dia;
- Consultar todos os Exames realizados num determinado dia;
- Consultar todos os Utentes com leituras de imc acima da média;
- Consultar todas o histórico de prescrições médicas por nome e data de início da toma;
- Consultar o histórico de Consultas de cada Médico.

```
public interface MedicoInterface extends Remote {

    public void save(String u, String r, String m, String f, String
df, String c, String e, String p) throws RemoteException;

    public void adicionarDadosFisiologicosUt(int idUtente, LocalDate
dataLeitura, double pressaoArterial, double temperaturaCorporal, double
saturacaoO2, double frequenciaCardiaca, double niveisColesterol,
double peso, double altura) throws RemoteException;

    public void adicionarExameUt(int idUtente, LocalDateTime dataExame,
String tipoExame, String descricao) throws RemoteException;

    public void adicionarPrescricaoUt(int idUtente, String
medicamento, LocalDate dataInicioToma, String duracao, String tomas,
int idmedico) throws RemoteException;

    public void adicionarConsultaUt(int idConsulta, int idUtente,
LocalDateTime dataConsulta, String tipoConsulta, int idMedico, String
observacoes) throws RemoteException;

    public String consultarUtente(int idUtente) throws
RemoteException;
    public String consultarConsultaUtente(int idUtente) throws
RemoteException;
    public String consultarExamesUtente(int idUtente) throws
RemoteException;
    public String consultarPrescricoesUtente(int idUtente) throws
RemoteException;
    public String consultarDadosFisiologicosUtente(int idUtente)
throws RemoteException;
    public String consultarFMedicaUtente(int idUtente) throws
RemoteException;

    public String editaConsulta(int idConsulta, int idUtente, String
dataNova, String idMedico, String observacoes) throws RemoteException;
```

```
    public String editaPrescricao (int idUtente, String
medicamentoAnt, LocalDate dataInicioToma, String medicamentoNovo,
String duracao,String tomas, String idmedico) throws RemoteException;

public String editaExame(int idUtente, LocalDateTime dataEAntiga,
String dataENova,String tipoExame, String descricao) throws
RemoteException;

    public String listarUtentes() throws RemoteException;
    public String listarFamiliares() throws RemoteException;
    public String listarFichasMedicas() throws RemoteException;
    public String listarDadosFisiologicos() throws RemoteException;
    public String listarExames() throws RemoteException;
    public String listarPrescricoes() throws RemoteException;
    public String listarConsultas() throws RemoteException;
    public String medicamentos () throws RemoteException;
    public String listarFamPUT (int idUtente) throws RemoteException;

    public int numeroTotalConsultas() throws RemoteException;
    public int numeroConsultasDia(LocalDateTime data) throws
RemoteException;
    public List<Exame> examesDia(LocalDateTime data) throws
RemoteException;
    public String utentesObesos() throws RemoteException;
    public String prescricaoMed (String medicamento, LocalDate dataP)
throws RemoteException;
    public String consultasPMed (int idMedicoP) throws
RemoteException;

}
```

*Figura 4-Interface do Medico.*

Por fim, a interface do **Rececionista Hospitalar**, permite realizar as seguintes operações:

- Salvar as alterações efetuadas;
- Registar Utentes, Médicos, Familiares e Rececionistas Hospitalares;
- Adicionar consultas à ficha médica de um determinado utente;
- Consultar informações pessoais de um determinado utente;
- Consultar histórico de Exames, Dados Fisiológicos, Consultas e Prescrições de um determinado utente;
- Consultar Ficha Médica de um determinado utente;
- Editar informações pessoais de um determinado utente;
- Editar consultas um determinado utente;
- Consultar todos os Utentes, Familiares e Fichas Médicas e Rececionistas existentes no sistema, bem como todas os Dados Fisiológicos, Exames, Prescrições e Consultas.
- Consultar o número total de consultas dadas por cada Médico;
- Consultar o número de consultas efetuadas num determinado dia;



- Consultar todos os exames realizados num determinado dia;
- Consultar todos os utentes com leituras de *imc* acima da média;
- Consultar todos os Familiares de um determinado Utente.

```
public interface RececionistaInterface extends Remote {

    public void save(String u, String r, String m, String f, String
df, String c, String e, String p) throws RemoteException;

    public void registaUtente(int idUtente, String cc, String nome,
String niss,String dataNascimento, String genero, String contacto,
String email) throws RemoteException;

    public void registarMedico(int idMedico, String cc, String nome,
String morada, String contacto, String email,String especialidade)
throws RemoteException;

    public void registarFamiliar(int idUtilizador, String nome, String
parentesco, int idUtenteAssociado) throws RemoteException;

    public void registarRececionista(int idRececionista, String cc,
String nome, String morada, String contacto, String email, String
departamento) throws RemoteException;

    public void adicionarConsultaUt(int idConsulta, int idUtente,
LocalDateTime dataConsulta, String tipoConsulta, int idMedico, String
observacoes) throws RemoteException;

    public String consultarUtente(int idUtente) throws
RemoteException;
    public String consultarConsultaUtente(int idUtente) throws
RemoteException;
    public String consultarPrescricoesUtente(int idUtente) throws
RemoteException;
    public String consultarExamesUtente(int idUtente) throws
RemoteException;
    public String consultarDadosFisiologicosUtente(int idUtente)
throws RemoteException;
    public String consultarFMedicaUtente(int idUtente) throws
RemoteException;

    public String editaUtente(int idUtente, String contacto, String
email) throws RemoteException;

    public String editaConsulta(int idConsulta, int idUtente, String
dataNova, String idMedico, String observacoes ) throws RemoteException;

    public String listarUtentes() throws RemoteException;
    public String listarFamiliares() throws RemoteException;
    public String listarMedicos() throws RemoteException;
    public String listarFichasMedicas() throws RemoteException;
    public String listarDadosFisiologicos() throws RemoteException;
    public String listarExames() throws RemoteException;
    public String listarPrescricoes() throws RemoteException;
    public String listarConsultas() throws RemoteException;
    public String listarRececionista() throws RemoteException;
```

```
public int numeroTotalConsultas() throws RemoteException;
public int numeroConsultasDia(LocalDateTime data) throws
RemoteException;
public List<Exame> examesDia(LocalDateTime data) throws
RemoteException;
public String utentesObesos() throws RemoteException;
public String listarFamPUt (int idUtente) throws RemoteException;
}
```

Figura 5- Interface do Rececionista Hospitalar.

Na secção seguinte, mostra-se como foram implementadas estas interfaces, isto é, como foram implementados os métodos ditados pelas interfaces apresentadas.

## 4. Implementação da Aplicação

### 4.1. Implementação das classes Utente, Médico, Rececionista e Ficha Médica

Como referido anteriormente, foram criadas classes para representar os utilizadores **Utente**, **Familiar**, e **Profissionais de Saúde**, nomeadamente o **Médico** e **Rececionista**. Foram também criadas classes para representar as consultas, exames, prescrições, leituras de dados fisiológicos e ainda a classe **Ficha Médica** para concentrar os dados de cada utente.

Em cada uma destas classes, foram definidos os atributos já mostrados no esquema da Figura 1 e ainda desenvolvidos os métodos *gets/sets* para estas variáveis poderem ser acedidas bem como atualizadas.

Na secção Arquitetura da Aplicação, foram mostradas as relações de cardinalidade entre as entidades desenvolvidas. Assim, como já mostrado anteriormente, a **Ficha Médica** apesar de ter um único utente associado, pode ter várias consultas, leituras de dados fisiológicos, prescrições e exames associados.

Para então armazenar os vários objetos **Consulta**, **DadosFisiológicos**, **Prescrição** e **Exame**, foram criados, em cada ficha médica, os dicionários, ou *HashMaps*, consultas, dadosFisiológicos, prescrições e exames, como mostrado na Figura 6.

No caso dos dicionários relativos às consultas, exames e dados fisiológicos, as chaves remetem às datas em que estes foram realizados, enquanto o valor trata-se do objeto e da informação dos atributos que o contém.

Quanto ao dicionário que armazena os dados das prescrições do utente, este está organizado através do nome do medicamento prescrito, isto é, as chaves do dicionário são *Strings* de nomes de medicamentos. Já os valores associados a estas chaves são listas de objetos Prescrição com as informações relativas a cada prescrição médica receitada.

É importante também referir que à **Ficha Médica** está também associado o objeto **Utente** com as informações do respetivo utente.

```
public class FichaMedica {  
  
    private Utente utente;  
    private Map<LocalDate, DadosFisiologicos> dadosFisiologicos;  
    private Map<LocalDateTime, Exame> exames;  
    private Map<String, List<Prescricao>> prescricoes;  
    private Map<LocalDateTime, Consulta> consultas;  
  
    ...  
}
```

**Figura 6** – Definição dos variáveis *utente*, *dadosFisiologicos*, *exames*, *prescrições* e *consulta* na *Ficha Médica* do *Utente*.

De forma semelhante às classes referidas anteriormente, foram desenvolvidos os métodos *gets* e *sets* para estas variáveis.

Para poder adicionar informação a estas estruturas de dados foram criadas funções que permitem adicionar prescrições, consultas, exames e leituras de dados fisiológicos. Todas estas funções funcionam de forma semelhante, criando o objeto respetivo e povoando-o com as informações necessárias, adicionando-o posteriormente ao dicionário adequado, como se irá explicar na secção seguinte.

#### 4.2. Implementação das Interfaces – Gestor Clínico

Para implementar as interfaces mostradas anteriormente, bem como auxiliar a gestão das estruturas de dados do objeto **Ficha Médica**, foi criada a classe **GestorClinico**.

Da mesma forma que a classe **Ficha Médica**, foram criados dicionários para armazenar todas as informações existentes, como se pode verificar na Figura 7. Para o dicionário de todos utentes, por exemplo, os dados são armazenados através de uma chave *idUtente*. O mesmo acontece para os dicionários dos médicos, familiares e rececionistas.

No caso dos exames e consultas, de forma semelhante às estruturas da classe **Ficha Médica**, as chaves dos dois dicionários correspondem às datas nas quais os exames e consultas foram efetuados, de forma a obter uma lista de dados de consultas ou exames que ocorrem num certo dia. É importante ressaltar que apesar do tipo de dados corresponder a *LocalDateTime*, no instante de inserção de dados nos dicionários, a data e hora introduzida foram manipuladas de forma a obter as horas, minutos e segundos no valor 0.

Para o caso do *dicTPrescricoes*, o objetivo foi armazenar as prescrições através de uma chave correspondente ao medicamento prescrito. Assim, o valor de cada chave, corresponde a uma lista

com todas as prescrições desse medicamento, independentemente do utente para o qual foi passada essa prescrição.

Para o dicionário com todos os dados fisiológicos, procedeu-se à organização de uma lista com todas as recolhas de dados fisiológicos de cada utente, sendo associadas como chaves os *ids* dos utentes.

```
public class GestorClinico extends UnicastRemoteObject implements
FamiliarInterface, UtenteInterface, RececionistaInterface,
MedicoInterface {
    public Map<Integer, Utente> dicTUtentes;
    private Map<Integer, Medico> dicTMedicos;
    private Map<Integer, Familiar> dicTFamiliares;
    private Map<LocalDateTime, List<Exame>> dicTExames;
    private Map<Integer, List<DadosFisiologicos>> dicTDadosF;
    private Map<String, List<Prescricao>> dicTPrescricoes;
    private Map<LocalDateTime, List<Consulta>> dicTConsultas;
    private Map<Integer, FichaMedica> dicTFichasMedicas;
    private Map<Integer, RececionistaHospitalar> dicTRececionista;
```

**Figura 7** – Definição dos variáveis *dicTUtentes*, *dicTMedicos*, *dicTFamiliares*, *dicTFamiliares*, *dicTFichasMedicas*, *dicTExames*, *dicTDadosF*, *dicTPrescricoes* e *dicTConsultas* na classe *GestorClinico*.

De forma a implementar os métodos referidos na interface, foram criadas diferentes funções para a manipulação de dados, de forma a conseguir consultar informações específicas, estatísticas, bem como alterar informações.

Ainda que as interfaces definam diferentes métodos, estes podem ser implementados no mesmo gestor para serem evocados, posteriormente, nos clientes que implementam as diferentes interfaces.

Desta forma, as funções desenvolvidas vão de encontro às mostradas nas Figuras 2, 3, 4 e 5. De forma sucinta foram desenvolvidos métodos que permitem:

- Listar todas as informações dos Utentes, Médicos, Rececionistas, Familiares, Prescrições, Consultas, Leituras de DadosFisiológicos e Exames;
- Registrar novos Utentes, Médicos, Rececionistas, Familiares e novas Prescrições, Consultas, Leituras de DadosFisiológicos e Exames para um dado utente (Registando não só nos dicionários do **GestorClinico** mas também nos dicionários adequados da Ficha Médica de cada Utente);
- Consultar os dados de um Utente, nomeadamente a sua Ficha Médica, Consultas, Exames, Prescrições, Leituras de Dados Fisiológicos;
- Consultar o Utente, Ficha Médica e Consultas por parte de um familiar (sendo feita uma verificação para averiguar a existência de autorização para essa consulta de dados);
- Alteração de dados do Utente, Consultas e Exames;
- Listagem de doentes Obesos;

- Exames num dia específico;
- Listar as consultas por médico;
- Número de Prescrições de um medicamento específico por data de início de toma;
- Listar os familiares associados a um utente específico;
- Carregar dados a partir de um *csv* e armazená-los no **GestorClinico**;
- Guardar os dados alterados num *csv*.

Para permitir editar registos dos utentes ou consultas, foi utilizada uma estratégia para minimizar o número de funções *editar*. No caso do Utente, por exemplo, foram escolhidos, em primeiro lugar, os campos passíveis de serem editados no objeto nomeadamente, o email e contacto, e desenvolvida uma única função *editaUtente*. Assim, caso seja necessário editar estes campos, basta apenas introduzir como parâmetros as novas informações.

Caso apenas seja necessário alterar um dos campos do objeto Utente, a nova informação é introduzida como parâmetro, e o outro parâmetro da função é preenchido com “\*”, que posteriormente, será interpretado pelo método como significativo de nenhuma mudança.

É importante também realçar que esta classe **GestorClinico** vai situar-se no servidor. De forma a poderem ser acedidos todos os dados em cada cliente, bem como as alterações efetuadas num dos clientes, foi necessário fazer referência à classe *UnicastRemoteObject*, isto é, fazer *extend* desta classe. É de notar também que em todos os métodos do **GestorClinico** teve de estar presente a exceção *RemoteException*, por ser a única compatível com a classe *UnicastRemoteObject*.

Assim, a utilização desta classe associada à utilização do java RMI, permitiu manter toda a informação atualizada em todos os clientes.

### 4.3. Carregamento de Dados

O carregamento de dados necessários para o povoamento das estruturas de dados mencionadas acima, foi efetuado através de *datasets* em formato *csv*. Foram então criados 8 *datasets* com informações relativas aos utentes, rececionistas, médicos, familiares, consultas, exames, prescrições e leituras de dados fisiológicos. Ainda que nos objetos consulta, exames, prescrições, dados fisiológicos e familiares esteja associado o objeto Utente associado, nos *datasets* criados estava apenas presente o id desse utente.

Assim, aquando do carregamento de dados foi necessária a criação dos objetos respetivos bem como a associação ao objeto utente, para todas estas informações ficarem interligadas a partir de objetos e não apenas a partir de referências a atributos desse objeto. No caso dos *datasets* relativos às consultas e prescrições, também se apresentava apenas o id do médico associado, sendo então

necessário proceder à associação de todo o objeto médico, povoado com as informações do médico com o id apresentado.

Posteriormente, para realizar o armazenamento dos dados alterados foi necessário efetuar o processo inverso, de forma a manter a coerência nos futuros carregamentos de dados do servidor. De forma mais detalhada, nos objetos onde se encontravam associados o objeto utente, por exemplo o objeto consulta, foi necessário extrair o id do utente através do método *getIdUtente*, armazenando então em cada linha, as informações do objeto e apenas o id do Utente. O mesmo aconteceu no caso dos médicos presentes nas consultas e prescrições.

Desta forma, foi possível manter a coerência entre os *datasets* originais, os métodos de carregamento criados e os futuros datasets armazenados.

## 5. Interação Servidor – Cliente

De modo a distribuir os serviços e recursos, aumentar a escalabilidade da aplicação, controlar o acesso e segurança desta, assim como garantir um maior controlo sobre o sistema, é implementada neste projeto uma arquitetura Cliente-Servidor, desenvolvendo-se para tal, um servidor e 4 clientes, um para cada interface apresentada anteriormente.

No servidor, começou-se pelo carregamento dos dados, inicialmente guardados em ficheiros *csv*, tal como pode ser verificado na Figura 8. Foi escolhido efetuar este passo no servidor, de modo que as alterações feitas por um cliente sejam atualizadas para todos os outros, garantindo assim a coerência das informações.

```
try {  
    GestorClinico server = new GestorClinico();  
    System.out.println(server.loadDados(  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetUtentes.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetFamiliares.csv" ,  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetMedicos.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetRececionistas.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetDadosFisiologicos.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetExames.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetConsultas.csv",  
        "C:/Users/maryy/Downloads/teste(1)/teste/DatasetPrescricoes.csv"));  
}
```

Figura 8- Carregamento dos dados no servidor.

É estabelecida, posteriormente a ligação de cada interface ao servidor, através da porta 50001, podendo, deste modo, serem invocados métodos para manipulação dos objetos associados às mesmas (Figura 9).

```
RececionistaInterface server4= (RececionistaInterface) server;
MedicoInterface server2= (MedicoInterface) server;
FamiliarInterface server1 = (FamiliarInterface) server;
UtenteInterface server3 = (UtenteInterface) server;
Naming.rebind("rmi://localhost:50001/gc3", server3);
Naming.rebind("rmi://localhost:50001/gc1", server1);
Naming.rebind("rmi://localhost:50001/gc2", server2);
Naming.rebind("rmi://localhost:50001/gc4", server4);

System.out.println("Running");

    } catch (RemoteException | MalformedURLException e) {
        throw new RuntimeException(e);
    }
}
```

Figura 9- Estabelecimento da ligação entre o servidor e as interfaces.

Relativamente ao lado do cliente, é efetuado, no início, o pedido de conexão ao servidor recorrendo ao método *lookup* da classe de Java RMI *Naming*, sendo passado como parâmetro a porta utilizada para comunicação com o servidor (Figura 10). Caso este processo seja bem sucedido, leva à apresentação de um menu ao utilizador, permitindo que o mesmo insira o valor referente à ação que pretende efetuar.

```
UtenteInterface ui = null;
try {
    ui = (UtenteInterface) Naming.lookup("rmi://localhost:50001/gc3");
} catch (NotBoundException e) {
    throw new RuntimeException(e);
} catch (MalformedURLException e) {
    throw new RuntimeException(e);
}
```

```
    } catch (RemoteException e) {  
        throw new RuntimeException(e);  
    }
```

Figura 10- Pedido de conexão ao servidor.

Para cada uma das opções é solicitado ao utilizador que insira os parâmetros necessários, se o método a implementar assim o exigir, executando o método presente na interface referente em seguida.

Quando a opção selecionada pelo utilizador é a 29, ou seja, sair da aplicação, é questionado a este se deseja guardar as alterações efetuadas. Tal sucede se for inserida a *string* “S” no terminal, chamando o método associado ao *save*, para cada um dos *datasets* necessários. Caso contrário, procede-se à saída da aplicação. Na figura seguinte, mostra-se o menu de funcionalidades desenvolvido (Figura 11).

```
Scanner scanner = new Scanner(System.in);  
  
int opcao;  
int idUtente;  
  
do {  
    System.out.println("==== Menu de Funcionalidades ====");  
    System.out.println("1. Consultar Utente");  
    System.out.println("2. Consultar Consulta do Utente");  
    System.out.println("3. Consultar Prescrições do Utente");  
    System.out.println("4. Consultar Exames do Utente");  
    System.out.println("5. Consultar Dados Fisiológicos do Utente");  
    System.out.println("6. Consultar Ficha Médica do Utente");  
    System.out.println("7. Consultar todos os Familiares associados a um Utente");  
    System.out.println("8. Sair");  
    System.out.print("Escolha uma opção: ");  
  
    opcao = scanner.nextInt();  
  
    switch (opcao) {  
        case 1:
```



```
        System.out.print("Digite o seu ID de Utente: ");
        idUtente = scanner.nextInt();
        System.out.println(ui.consultarUtente(idUtente));
        break;
    case 2:
        System.out.print("Digite o seu ID de Utente: ");
        idUtente = scanner.nextInt();

        System.out.println(ui.consultarConsultaUtente(idUtente
        ));
        break;
    case 3:
        System.out.print("Digite o seu ID de Utente: ");
        idUtente = scanner.nextInt();

        System.out.println(ui.consultarPrescricoesUtente(idUt
        ente));
        break;
    case 4:
        System.out.print("Digite o seu ID de Utente: ");
        idUtente = scanner.nextInt();

        System.out.println(ui.consultarExamesUtente(idUtente
        ));
        break;
    case 5:
        System.out.print("Digite o seu ID de Utente: ");
        idUtente = scanner.nextInt();

        System.out.println(ui.consultarDadosFisiologicosUtent
        e(idUtente));
        break;
    case 6:
        System.out.print("Digite o seu ID do Utente: ");
        idUtente = scanner.nextInt();

        System.out.println(ui.consultarFMedicaUtente(idUtente
        ));
        break;
    case 7:
```

```
        System.out.print("Digite o seu ID do Utente: ");
        idUtente = scanner.nextInt();
        System.out.println(ui.listarFamPUt(idUtente));
        break;
    case 8:
        System.out.println("Saiu do Sistema");
        break;
    default:
        System.out.println("Opção inválida. Tente novamente.");
    }
    System.out.println();

    }
    while (opcao!= 8);
    scanner.close();
}
}
```

**Figura 11-** Menu do cliente Utente e suas opções.

## 6. Conclusão

Com o desenvolvimento deste projeto foi possível perceber de um modo mais claro e prático como funciona a arquitetura Cliente-Servidor e como é desenvolvida, o modo como é estabelecida a conexão entre as duas partes e como é efetuada a manipulação de objetos e as suas permissões para cada tipo de cliente.

Por último, é importante referir que alguns dos trabalhos futuros a implementar neste projeto seriam o desenvolvimento de mais e variados métodos de modo a obter-se uma aproximação maior às circunstâncias reais, bem como o desenvolvimento de uma interface gráfica permitindo um acesso mais simplificado por parte de cada cliente.

Adicionalmente, identificam-se as necessidades de otimização dos algoritmos de procura implementados nos métodos aqui presentes, a visualização de estatísticas em formato de gráfico, sendo mais fácil a compreensão da informação que consta nos ficheiros *csv*, assim como a implementação de métodos de remoção de registos.